

14

Packaging and Deploying WinUI Applications

WinUI developers have several options to package and deploy their applications. Developers can create an account on **Microsoft Store** and upload a packaged app to be released for public consumption through **Microsoft Partner Center**. App packages can also be created to be distributed by organizations through **Microsoft Endpoint Manager** and **Microsoft Intune**, or sideloaded by individuals on Windows PCs.

In this chapter, we will cover the following topics:

- Discovering application packaging and **MSIX** basics
- Getting started with application packaging in Visual Studio
- Deploying applications with **Windows Package Manager**
- Distributing applications with the Microsoft Store
- Sideloaded WinUI applications with MSIX

By the end of this chapter, you will understand the methods available to package and distribute your WinUI applications and how to use each of them.

Technical requirements

To follow along with the examples in this chapter, the following software is required:

- Windows 10 version 1809 (build 17763) or newer, or Windows 11
- Visual Studio 2022 or later, with the **.NET Desktop Development** workload configured for Windows App SDK development

The source code for this chapter is available on GitHub at this URL:

<https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/master/Chapter14>.

Discovering application packaging and MSIX basics

For most of this book, we have built and run our WinUI 3 applications locally. Now, it's time to learn the concepts of packaging and deploying WinUI applications and put that knowledge to use.

Why package your application? Well, an application package is the easiest way for WinUI applications and their dependencies to be installed in Windows. Today, when you run a WinUI project in Visual Studio, the **integrated development environment (IDE)** creates a package and deploys it locally. Packaging serves several other important purposes, outlined as follows:

- **Providing a clean uninstall:** A packaging system ensures that any files installed or updated with an application are removed or restored to their previous state when the application is uninstalled
- **Bundling dependencies:** The application package will bundle and deliver all your application's dependencies, optimizing disk space by sharing files across installed applications when possible
- **Facilitating updates:** Differential updates are optimized to only deliver files that require updating, based on the manifest of the original package and the updated one
- **Declaring capabilities:** By declaring your application's capabilities in the manifest, users know what types of access your app requires before they choose to install it
- **Verifying integrity and authenticity:** In order to install a Windows application on someone else's device, the application package must be digitally signed with a valid certificate from a **trusted signing authority**

If you are going to distribute your WinUI applications, the packaging format you will use is **MSIX**. What is MSIX? Let's find out.

MSIX

MSIX is the latest standard introduced by Microsoft to package applications. It is not only for Windows. The **MSIX SDK** (<https://learn.microsoft.com/windows/msix/msix-sdk/sdk-overview>) is an open source project that can be used to create application packages for any platform. You can use the SDK for Windows, Linux, macOS, iOS, Android, and even web browsers. We will focus on delivering WinUI applications to Windows users in this chapter, but you can learn more about the MSIX SDK on its GitHub repository at <https://github.com/Microsoft/msix-packaging>.

On Windows, the following platforms currently support the MSIX format:

- Windows 10, version 1709 and later, and Windows 11
- Windows Server 2019 **Long-Term Servicing Channel (LTSC)** and later
- Windows Enterprise 2019 LTSC and later

Earlier versions of Windows 10 required **APPX** packages, which were the predecessor to MSIX packages. However, all Windows 10 versions supported by WinUI 3 also support MSIX packages. Microsoft introduced MSIX packages in 2018 as an evolution of APPX, intending to fill the needs of APPX as well as legacy **Windows Installer (MSI)** packages. MSIX is an open standard, and as such, it can be used to distribute applications to any platform. MSI has been the standard to package and install Windows

desktop applications since 1999. Installing Windows applications with MSI packages is supported in Windows 95 and later operating systems.

With the new MSIX packaging standard, **Universal Windows Platform (UWP)** applications delivered to Windows users run inside a lightweight app container. WinUI 3 applications can also be configured to run in an app container for additional security. The **Windows App Container** provides a sandbox for the execution of an application, restricting access to the registry, filesystem, and other system capabilities, such as the camera. Any capabilities that an application needs to access must be specified in the manifest file. Applications packaged to run in an app container and installed with MSIX have *read access* to the Windows registry by default. In this configuration, any data written to the virtualized registry will be completely removed if the application is uninstalled or reset. The same is true of data written to the virtual filesystem.

As documented on Microsoft Learn

(<https://learn.microsoft.com/windows/msix/overview#inside-an-msix-package>), an MSIX package's contents are grouped into *application files* and *footprint files*, as illustrated in the following diagram:

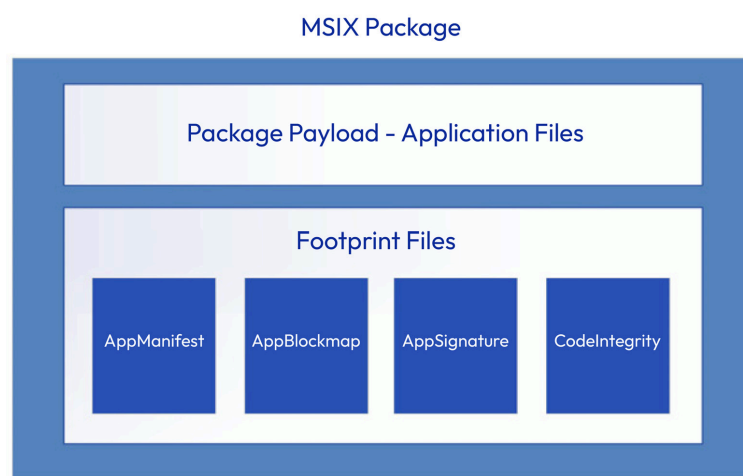


Figure 14.1 – The contents of an MSIX package

The application files are the payload of code files and other assets delivered to a user. The footprint files are the metadata and other resources that a package needs to ensure the application files are delivered as intended. This metadata includes the following:

- **AppManifest:** The manifest (**AppxManifest.xml**) includes information about the application's identity, dependencies, capabilities, extensibility points, and visual elements. This is generated from the **Package.appxmanifest** file in your WinUI project.
- **AppBlockmap:** The **AppxBlockMap.xml** file includes an indexed and cryptographically hashed list of the files in the package, digitally signed to ensure its integrity when the package is signed.
- **AppSignature:** The **AppxSignature.p7x** file in the package is generated when the package is signed. This allows the operating system to validate the signature during installation.

- **CodeIntegrity:** Code integrity is ensured by verifying information about the package in `AppxManifest.xml`, `AppxBlockMap.xml`, and `AppxSignature.p7x`.

The application files in the package will be installed to `C:\Program Files\WindowsApps \<package_name>`, with the application executable found at `C:\Program Files\WindowsApps\<package_name>\<app_name>.exe`. Note that you can't directly execute this EXE file, and access to the `WindowsApps` folder is restricted by Windows. Data created by the application during and after installation will be stored under `C:\Users\<user_name>\AppData\Local\Packages\<package_name>`. All the application files, dependencies, and data will be removed when the application is uninstalled.

Now that we know the background and history of MSIX, let's review a few of the tools available to developers and IT pros.

Reviewing MSIX tools and resources

Before we start using MSIX to package our own application, we will review a couple of other available tools and resources, as follows:

- **MSIX Toolkit:** The MSIX Toolkit is an open source collection of MSIX tools and scripts maintained by Microsoft on GitHub:
<https://github.com/microsoft/MSIX-Toolkit>.
- **MSIX Labs:** Microsoft maintains a set of hands-on tutorials for developers and IT pros interested in leveraging MSIX, to package and distribute their applications: <https://github.com/Microsoft/msix-labs>.
- **MSIX Packaging Tool:** The MSIX Packaging Tool is an application to repackage classic applications in the MSIX format. Existing EXE, MSI, and **Application Virtualization (App-V)** installation packages can be converted to MSIX packages with the tool via its interactive UI or command-line tooling. It is available on the Microsoft Store at <https://apps.microsoft.com/store/detail/msix-packaging-tool/9N5LW3JBCXKE>.
- **MSIX videos:** Microsoft Learn has a series of introductory videos on MSIX packaging here:
<https://learn.microsoft.com/windows/msix/resources#msix-videos>. This is a great way to get started on your MSIX journey.
- **MSIX community:** **Microsoft Tech Community** has a group of discussion spaces dedicated to MSIX packaging and deployment. Join the community and get involved here:
<https://techcommunity.microsoft.com/t5/msix/bd-p/MSIX-Discussions>.

These tools and resources will assist you in your journey while learning the ins and outs of WinUI application deployment. It is important to remember that MSIX is an area of continual investment by Microsoft. It is the go-forward strategy and recommendation to package all applications. WinUI developers don't need to have a deep understanding of MSIX. You will only need a basic knowledge of MSIX and the properties that are relevant to our applications. Before we get hands-on, we'll briefly discuss the

concepts of **packaged applications**, **package identity**, and **app containers** in Windows.

Packaged applications and application identity

There are a few deployment concepts that are specific to the Windows App SDK. Let's cover the basics here before we start to package and deploy our own applications.

One of the most important concepts to understand is packaged applications. The concepts we have discussed so far apply to packaged WinUI applications. These applications are packaged and installed with MSIX, they have package identity, and by default, they are **framework-dependent**. We just touched on some important concepts in that sentence; let's start by examining package identity.

All packaged applications, whether they are WinUI, UWP, or some other desktop Windows application type, benefit from package identity. Package identity is the unique identifier used by Windows to distinguish and validate the identity of your application. There are some Windows App SDK features that are only available to applications with package identity. The new Windows App SDK notifications APIs that we used in [*Chapter 8. Adding Windows Notifications to WinUI Applications*](#), are available to any packaged application because those apps have package identity. For a current list of other features that require package identity, see this Microsoft Learn topic:

<https://learn.microsoft.com/windows/apps/desktop/modernize/modernize-packaged-apps>.

Now, let's discuss packaged applications and **unpackaged applications**. We already reviewed some aspects of packaged apps. Most packaged apps are both packaged and installed with MSIX. However, there is a particular variation of a packaged application called a **packaged app with an external location**. These applications are packaged with MSIX, thus having package identity, but they are installed with a different installer mechanism. We won't deploy a packaged app with an external location, but you can read more about them on Microsoft Learn:

<https://learn.microsoft.com/windows/apps/desktop/modernize/grant-identity-to-nonpackaged-apps>.

Unpackaged applications don't have package identity, so there are limits on the APIs they can access, as previously discussed. In most cases, you will only choose this route for legacy applications that don't require any of the features that require package identity and often require fewer restrictions on their capabilities, such as filesystem or registry access. UWP applications cannot be unpackaged, but WinUI applications can be.

The final concept to review is that of framework-dependent applications and **self-contained** applications. WinUI applications that are framework-dependent rely on the Windows App SDK runtime to be installed on the target machine. This reduces the size of your application's installer and allows apps to receive security bug fix updates during runtime, without

each application updating. The downsides are having to check for the presence of the runtime during installation and the potential risk of users uninstalling the runtime after your application has been installed.

Self-contained deployments bundle the Windows App SDK runtime with the MSIX. This increases the size of the installer but provides complete control over the version of the runtime that the application uses. Self-contained applications also enable **Xcopy deployment**. You can copy the application and its dependencies from the project's output folder to any supported PC. Other than the package size, the primary downside to this option is performance. Self-contained applications are slower to load, and they use more system memory because their dependencies are not shared with other WinUI applications on the system. They are isolated to your application's process.

For more detailed information on these concepts, there are two excellent topics on Microsoft Learn:

- The *Windows App SDK deployment overview* (<https://learn.microsoft.com/windows/apps/package-and-deploy/deploy-overview>) discusses the pros and cons of self-contained deployment
- The *Deployment overview* topic (<https://learn.microsoft.com/windows/apps/package-and-deploy/>) dives deeper into packaged apps versus unpackaged apps

Now, let's get started and go hands-on, creating our MSIX package in Visual Studio.

Getting started with application packaging in Visual Studio

In this section, we'll see how we can package our applications with MSIX in Visual Studio. Visual Studio 2022 includes two WinUI project templates capable of creating MSIX deployment packages, outlined as follows:

- **Blank App, Packaged (WinUI 3 in Desktop)**: Creates a WinUI 3 project with a `package.appxmanifest` file to generate an MSIX package
- **Blank App, Packaged with Windows Application Packaging Project (WinUI 3 in Desktop)**: Creates a solution with two projects – a WinUI 3 project and a packaging project that contains the `package.appxmanifest` file

We will work with the completed **MyMediaCollection** solution from [Chapter 8](#). You can either use your own solution from that chapter or download a copy from this chapter's GitHub repository at <https://github.com/PacktPublishing/Learn-WinUI-3-Second-Edition/tree/master/Chapter14>. Let's see how to generate an MSIX package for the application, as follows:

1. Start by opening the solution in Visual Studio.

2. If you want to review the manifest data for the project, you can open the **Package.appxmanifest** file and review the settings on each tab, as illustrated in the following screenshot:

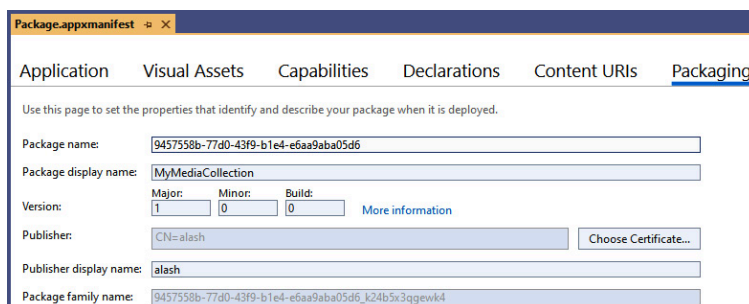


Figure 14.2 – Reviewing the application manifest

3. Then, right-click the **MyMediaCollection** project in **Solution Explorer** and select **Package and Publish | Create App Packages**. The **Create App Packages** window will appear, as illustrated in the following screenshot:

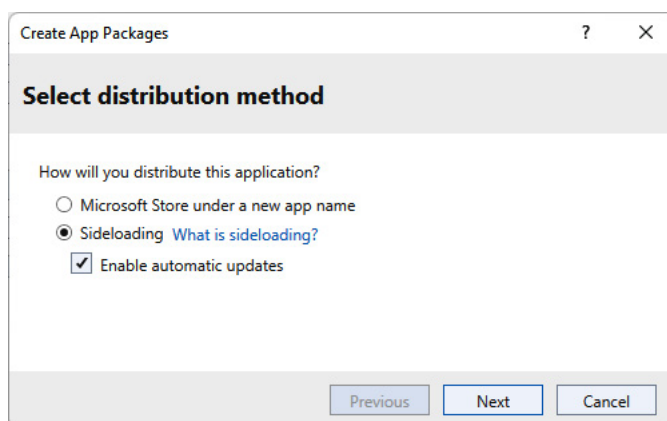


Figure 14.3 – The Create App Packages window

4. Select **Sideload**, leave **Enable automatic updates** selected, and click **Next**. This is the MSIX equivalent of **ClickOnce deployment** (<https://learn.microsoft.com/visualstudio/deployment/clickonce-security-and-deployment>).
5. On the **Select signing method** page, remove the current certificate if one appears. You will see options to add a certificate from **Azure Key Vault**, the Store, or a local file, or create a new one, as illustrated in the following screenshot:

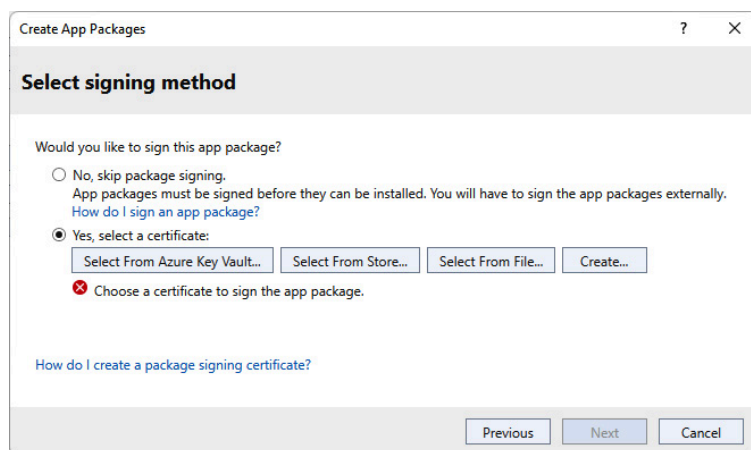


Figure 14.4 – Selecting a signing method for the package

6. We will create a self-signed certificate here. If you wanted to create a package with a certificate from a trusted authority, you would choose one of the **Select** buttons here to import the **.pfx** file. For this exercise, select **Create**.
7. Enter and confirm a secure password on the **Create a Self-Signed Test Certificate** dialog, and click **OK**. If you are prompted to overwrite an existing certificate, you can do that. Select **Yes** to continue. You will also receive a message about the certificate being imported to the certificate store for package signing. You can click **OK** on that message dialog.
8. When you return to the **Select signing method** page, the new certificate details will appear. Make sure you click the **Trust** button on the page to trust the certificate on your machine before continuing. Click **Next**.
9. On the **Select and configure packages** page, there are options to change the version number, generate an app bundle, or add, remove, and change solution configuration mappings. There is also a checkbox to include public symbol files with your package. Leave all the defaults and click **Next**.
10. On the **Configure update settings** page, an **Installer location** path is required. For applications where you will install a workstation on an internal network, you could enter the network path to the package here. For our test purposes, we will enter a local path, as illustrated in the following screenshot. For an application to update correctly on other machines, the same local path to the installer would need to exist:

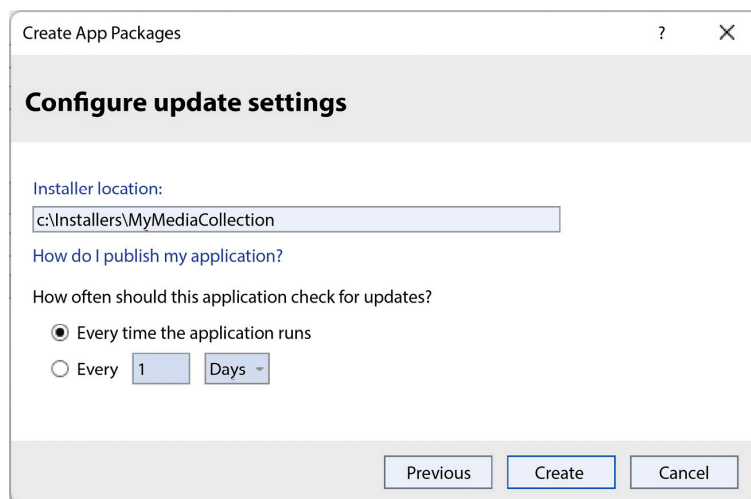


Figure 14.5 – Configuring update settings for the package

11. Click **Create**. The solution will be built, and the package will be created. When it completes, click the **Copy and Close** button to copy the installer to your selected installer location.
12. Open the **MyMediaCollection_1.0.0.0_x64_Debug_Test** folder to see the generated MSIX file, as illustrated in the following screenshot:

Name	Date modified	Type	Size
Add-AppDevPackage.resources	9/23/2023 10:48 AM	File folder	
Dependencies	9/23/2023 10:48 AM	File folder	
Add-AppDevPackage	7/21/2023 6:29 PM	Windows PowerShell Script	37 KB
Install	7/21/2023 6:29 PM	Windows PowerShell Script	14 KB
MyMediaCollection_1.0.0.0_x64_Debug	9/23/2023 10:43 AM	Security Certificate	1 KB
MyMediaCollection_1.0.0.0_x64_Debug	9/23/2023 10:43 AM	MSIX File	41,563 KB
MyMediaCollection_1.0.0.0_x64_Debug.msixsym	9/23/2023 10:43 AM	MSIXSYM File	32 KB

Figure 14.6 – The MSIX package created for MyMediaCollection

13. Navigate to the **MyMediaCollection** parent folder and double-click the **index.html** file to install the package. It will open the file in your default browser and present you with options to install any of the configurations selected during packaging. Note, in the following screenshot, I packaged the application a second time, so the version number has been automatically incremented to **1.0.1.0**:

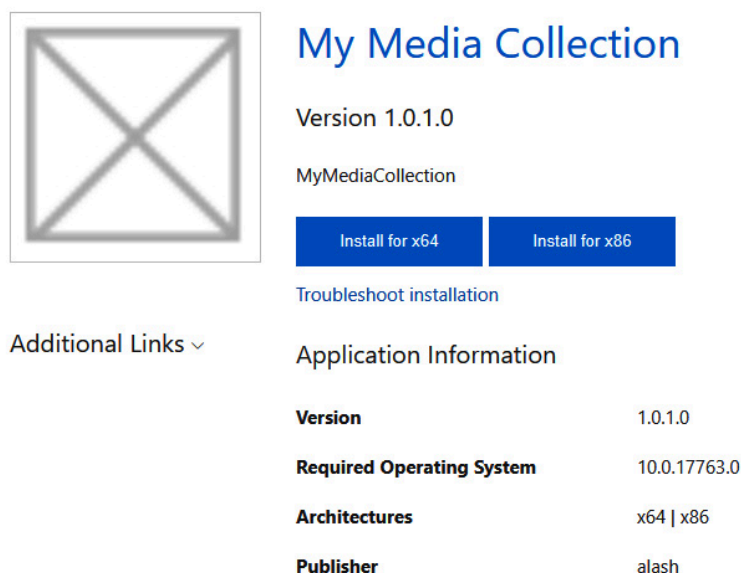


Figure 14.7 – The installation page for MyMediaCollection

14. If you try to install the package on the same machine where it has already been installed and run, you will be prompted to update the application. Continue to install or update the application on your system.

Creating a package with Visual Studio is how most IDE users will choose to generate their installers. There are also tools to create MSIX packages and bundles at the command line, which are described in the Microsoft Learn documentation:

<https://learn.microsoft.com/windows/msix/package/manual-packaging-root>.

Now, let's look at an alternative distribution method, Windows Package Manager.

Deploying applications with Windows Package Manager

Windows Package Manager, also referred to by its command name **WinGet**, is an open source command-line package management tool from Microsoft. In this section, we will cover how to use the **WinGet** command to install published packages and the steps to add your own MSIX packages to the **Windows Package Manager community repository**, which can be found here: <https://github.com/microsoft/winget-pkgs>. This is where **WinGet** finds available packages to install. The Microsoft Store is one of the package sources available to **WinGet**. So, if you plan to publish your application to the store, it is not necessary to also publish it to the WinGet repository.

Let's start by reviewing the steps to add a package to the community repository.

Adding a package to the community repository

To make your applications available to Windows users with the **WinGet** command, they must be published to Microsoft's Package Manager community repository or the Microsoft Store. Any application published to the community repository can be discovered and installed through the **WinGet** command in Windows.

NOTE

This method of distribution is inherently less secure than distribution through the Microsoft Store. In theory, anyone with access to the public repository could extract the package and de-compile your application.

To add your existing MSIX bundle to the repository, we will need to make it publicly available, create a WinGet manifest, and submit a GitHub **pull request (PR)** to have the manifest added to the community repository, as follows:

1. Start by pushing the contents of the installer folder we created (**C:\Installers\MyMediaCollection**) to a public URL. To do this, you could create a static website in Azure. This Microsoft Learn topic walks through the process: <https://learn.microsoft.com/azure/static-web-apps/get-started-portal>. This would require a GitHub or Azure DevOps repository to host the files to be deployed. The files can be made available on any public URL. Another option is to host the files in **Azure Blob Storage** (<https://learn.microsoft.com/azure/storage/blobs/storage-blob-static-website-how-to?tabs=azure-portal>). We will choose this option.
2. After the **Storage account** application resource has been created in Azure, open the account overview, and use the **Static website** page to enable this option. Enter the **index.html** filename that we'll upload as the index document name. After you save the file, the **Primary endpoint** field will appear, as seen in the following screenshot:

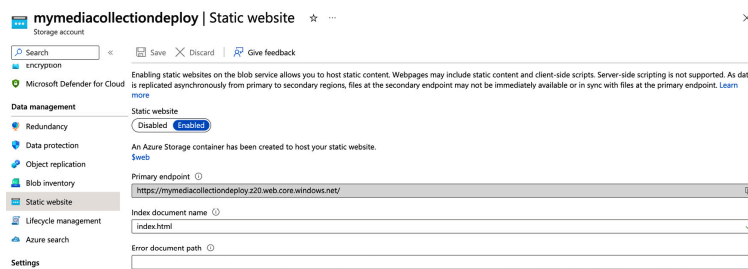


Figure 14.8 – Setting up a static website in Azure Blob Storage

3. Click the **\$web** link on that page to navigate to the portal page, where you can upload files to the static web app folder.
4. Click **Upload**, and drag and drop or browse for your files. Upload all of the files and folders to the **C:\Installers\MyMediaCollection** folder.

Once your package is in the cloud, you are ready to create your manifest file for the community repository. The file is created in the **YAML** format. YAML files are the current standard for **DevOps** workflows.

For detailed instructions on creating a manifest and links to learn more about YAML, check out this Microsoft Learn topic:

<https://learn.microsoft.com/windows/package-manager/package/manifest>:

1. Name your YAML file **YourCompany.MyMediaCollection.yaml**; the contents should look something like this:

```
PackageIdentifier: YourCompany.MyMediaCollection
PackageVersion: 1.0.0.0
PackageLocale: en-US
Publisher: Your Company Name
PackageName: My Media Collection
License: MIT
ShortDescription: My Media Collection helps you get your media collection organized.
Installers:
  - Architecture: x64
    InstallerType: msix
    InstallerUrl: https://mymediacollectiondeploy.z20.web.core.windows.net/MyMediaCollection_1.
    InstallerSha256: 712f139d71e56bfb306e4a7b739b0e1109abb662dfa164192a5cfd6adb24a4e1
    SignatureSha256: e53f48473621390c8243ada6345826af7c713cf1f4bbdf0d030599d1e4c175ea
ManifestVersion: 1.0.0
```

2. To get the **Sha256** information for your **.msix** file, you can use this command:

```
certUtil -hashfile
C:\Installers\MyMediaCollection\
MyMediaCollection_1.0.0.0_x64_Debug_Test\
MyMediaCollection_1.0.0.0_x64_Debug.msix SHA256
```

3. Then, test your manifest with WinGet. To install WinGet, you can install the **App Installer** app from the Microsoft Store at <https://apps.microsoft.com/store/detail/app-installer/9NBLGGH4NNS1>.
4. After the application is installed from the store, test the syntax in your manifest with the following command:

```
winget validate <manifest-file-name>
```

5. Then, test installing the app from your manifest with this command:

```
winget install -m <manifest-file-name>
```

6. If your tests succeed, you're ready to submit a PR to the community repository. Use the following path structure when you submit the PR. Your **publisher** name must be unique. If you have an account on **Microsoft Partner Center** to publish apps to the store, you should use the same publisher name here. **package** is the name of your application:

```
manifests\<letter of alphabet>\<publisher>\<package>\<version>\<filename>.yaml
```

NOTE

If you're unfamiliar with the GitHub forking and PR workflow, the Package Manager documentation has more detailed steps:

<https://learn.microsoft.com/windows/package-manager/package/repository#step-3-clone-the-repository>.

After the PR has been approved and merged, any users with WinGet will be able to install your application from the command line. Let's see how to use WinGet next.

Using WinGet for package management

WinGet is the command-line client for Windows Package Manager. If you are familiar with other application package managers, such as **Chocolatey** for Windows (<https://chocolatey.org/>) or **Homebrew** for macOS (<https://brew.sh/>), WinGet will feel familiar to you. A package manager allows you to install, list, and update applications on your operating system and to script the installation of multiple applications, such as when you're setting up a new computer. In this section, we will see how to use WinGet to install **Windows Terminal**, Microsoft's modern command-line app, built with WinUI!

NOTE

To read about how the Windows Terminal team created the application with WinUI, check out this blog post:

<https://devblogs.microsoft.com/commandline/building-windows-terminal-with-winui/>.

Let's now look at the steps:

1. You already have the App Installer preview installed, so open **Command Prompt**.
2. Test WinGet with the following command:

```
winget -?
```

The WinGet help and available commands should appear in your **Command Prompt** window.

3. Now, run the **search** command to find the application we want to install, as follows:

```
winget search windowsterminal
```

You will see two results, as illustrated here:

```
PS C:\Installers\MyMediaCollection\MyMediaCollection_1.0.1.0_x64_Debug_Test> winget search windowsterminal
Name      Id              Version        Source
-----
Windows Terminal Preview Microsoft.WindowsTerminal.Preview 1.17.10234.0 winget
Windows Terminal Microsoft.WindowsTerminal 1.16.10261.0 winget
PS C:\Installers\MyMediaCollection\MyMediaCollection_1.0.1.0_x64_Debug_Test> |
```

Figure 14.9 – Performing a search with WinGet

NOTE

You may have to agree to the Microsoft Store terms of transaction before continuing.

4. To get more information about the package, use the **show** command, as follows:

```
winget show Microsoft.WindowsTerminal
```

This will return the following from the application's manifest:

```
License: MIT
License Url: https://github.com/microsoft/terminal/blob/master/LICENSE
Privacy Url: https://privacy.microsoft.com
Copyright: Copyright (c) Microsoft Corporation. All rights reserved.
Copyright Url: https://github.com/microsoft/terminal/blob/master/LICENSE
Release Notes:
  This release of Windows Terminal addresses a crash in self-elevation.

  Huge thanks to @jboelter for fixing it in 1.17 (#14637).
Release Notes Url: https://github.com/microsoft/terminal/releases/tag/v1.16.10261.0
Tags:
  cli
  cmd
  command-line
  command-prompt
  console
  developer-tools
  powershell
  ps
  shell
  terminal
  utilities
  wsl
Installer:
  Installer Type: msix
  Installer Url: https://github.com/microsoft/terminal/releases/download/v1.16.10261.0/Microsoft.WindowsTerminal_Win10_1.16.10261.0_8wekyb3d8bbwe.msixbundle
  Installer SHA256: ba6fc6854e713894b4089cf2021e8b4887cf737ab4b9c4f9390462dd2708298
  Release Date: 2023-01-27
PS C:\Installers\MyMediaCollection\MyMediaCollection_1.0.1.0_x64_Debug_Test> |
```

Figure 14.10 – Viewing manifest information for a WinGet package

5. Then, enter this command to install Windows Terminal:

```
winget install Microsoft.WindowsTerminal
```

You should see a message that Windows Terminal was found and installed on your PC. If you already have Windows Terminal installed, you could try another app such as **Microsoft.PowerToys** (you can find out more about **Microsoft PowerToys** at

<https://learn.microsoft.com/windows/powertoys/>).

That's all there is to using WinGet. Because it is a command-line tool, you can build scripts that install all the software you need to get a new PC or **virtual machine (VM)** up and running. Get more information about scripting with WinGet on Microsoft Learn:

<https://learn.microsoft.com/windows/package-manager/winget/#scripting-winget>.

Let's move along to our final section, where we will learn about distributing applications in the Microsoft Store.

Distributing applications with the Microsoft Store

We have seen how to deliver WinUI applications to users through packages that can be sideloaded and with WinGet. There are a couple of other distribution channels available to Windows developers – **Microsoft Intune** for enterprise application distribution, and the **Microsoft Store** for consumer apps.

A deeper dive into **Microsoft Endpoint Configuration Manager** and Intune is beyond the scope of this book, but if you are interested in learning how to distribute **line of business (LOB)** applications through them,

you can read this Microsoft Learn topic:

<https://learn.microsoft.com/windows/apps/publish/distribute-lob-apps-to-enterprises>.

The Microsoft Store is the consumer app store for Windows users. The store accepts submissions for free and paid apps. Additional monetization options such as in-app purchases, sale pricing, and paid apps with a free trial period can also be configured.

In this section, we will cover the basics of submitting a free application to the store. If you want to learn more about monetizing your app, you can start here: <https://learn.microsoft.com/windows/apps/publish/publish-your-app/price-and-availability>.

Let's see how to submit **MyMediaCollection** to the Microsoft Store.

Preparing a free application for the Microsoft Store

In this section, we will use Visual Studio to prepare and submit the **MyMediaCollection** application to the Microsoft Store. The Microsoft Store is the primary delivery outlet for consumer applications. Before you start this process, you will need to have a developer account on the store. Let's do this now, as follows:

1. First, visit the sign-up page for the Microsoft Store (<https://developer.microsoft.com/microsoft-store/register/>) and click **SIGN UP**.
2. Sign in with a Microsoft account, and select the country or region where you or your company is located.

NOTE

This account will be the owner of the Store account and cannot easily be changed. If you create a Store account for an organization, it is recommended to use a separate Microsoft account that is not tied to any individual at the organization or company.

3. Choose **Individual** or **Company** as your account type.
4. Enter your publisher display name. This is the name that will be seen by users on your public application store listings, so choose this carefully.
5. Enter your contact details – this information will be used by Microsoft to contact you if there are any issues or updates with your store listings. Click **Next** to continue and enter payment information.
6. Pay a one-time store registration fee. Microsoft charges a small fee the first time you register for an account on the store. This helps to prevent fraudulent and malicious accounts from being created. In most countries, the fee is approximately *\$19 US Dollars (USD) for individuals* and *\$99 USD for a company account*.

The full listing of fees by country is available here:

<https://learn.microsoft.com/windows/apps/publish/partner->

[center/account-types-locations-and-fees#developer-account-and-app-submission-markets](#).

7. When you are finished, click **Review**.
8. Review the details for your account and the app developer agreement, and click **Finish** to confirm your registration. Payment will be processed at this time, and you will receive a confirmation email.

Now that you have an account on the Microsoft Store, we can proceed to submit our first app, as follows:

1. Return to Visual Studio and right-click on the **MyMediaCollection** project in **Solution Explorer**.
2. Select **Package and Publish** | **Create App Packages**. This process starts out the same as when creating a package for sideloading.
3. On the **Select distribution method** screen, select **Microsoft Store under a new app name**, and click **Next**.
4. On the **Select an app name** screen, ensure the Microsoft account linked to your store account is selected, and enter your desired application name in the **Reserve a new app name** field. Click **Reserve** to check whether the name is available. Each app name must be unique across the Microsoft Store. The app name will appear in your list of apps, as illustrated in the following screenshot:

The screenshot shows the 'Create App Packages' dialog box with the 'Select an app name' tab selected. At the top right, there is a dropdown menu for 'Microsoft account'. Below it, there is a checkbox labeled 'Include app names that already have packages' and a 'Refresh' link. A table lists 'Existing app names' with two columns: 'App Name' and 'Package Identity in the Microsoft Store'. The table contains five rows: 'Gu-Ha-Haa', 'My Media Collection', 'Newsworks Mobile', 'Project Tracker 2020', and 'Simple Task Tracker', all with 'None' in the second column. Below the table is a 'Reserve a new app name:' section with a text input field and a 'Reserve' button. At the bottom of the dialog are 'Previous', 'Next', and 'Cancel' buttons.

App Name	Package Identity in the Microsoft Store
Gu-Ha-Haa	None
My Media Collection	None
Newsworks Mobile	None
Project Tracker 2020	None
Simple Task Tracker	None

Figure 14.11 – Selecting an app name for the Microsoft Store listing

5. Select the app name in your **Existing app names** list, and click **Next**.
6. You can leave the default values on the **Select and configure packages** screen unless you want to update the version or the solution configuration mappings. Click **Create** at the bottom of the screen, as illustrated in the following screenshot:

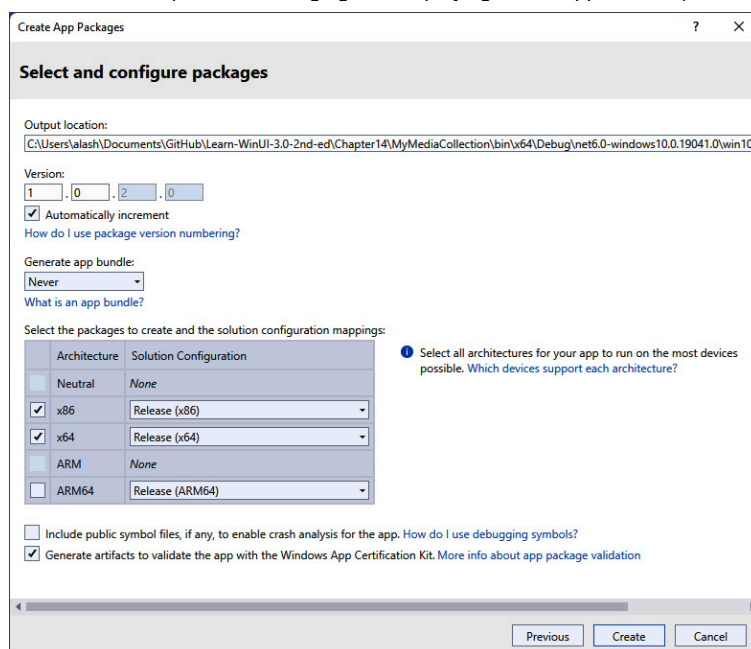


Figure 14.12 – The Select and configure packages screen

- Visual Studio will build your project and prepare the package for the store. It will also do some validation of your app manifest data. If you receive any errors during packaging, fix them and try again. If you feel your submission is ready to go after reviewing the submission checklist (<https://learn.microsoft.com/windows/apps/publish/publish-your-app/create-app-submission#app-submission-checklist>), you can check the **Automatically submit to the Microsoft Store after Windows App Certification Kit validation** checkbox on the **Finished creating package** screen. This will submit your package to the store upon a successful validation run.
- Click the **Launch Windows App Certification Kit** button. When the certification kit launches, keep all the tests selected, and then click **Next**. The tests will take several minutes to complete, and the application may launch several times during the validation process. The following screenshot illustrates the **Validating App** progress:

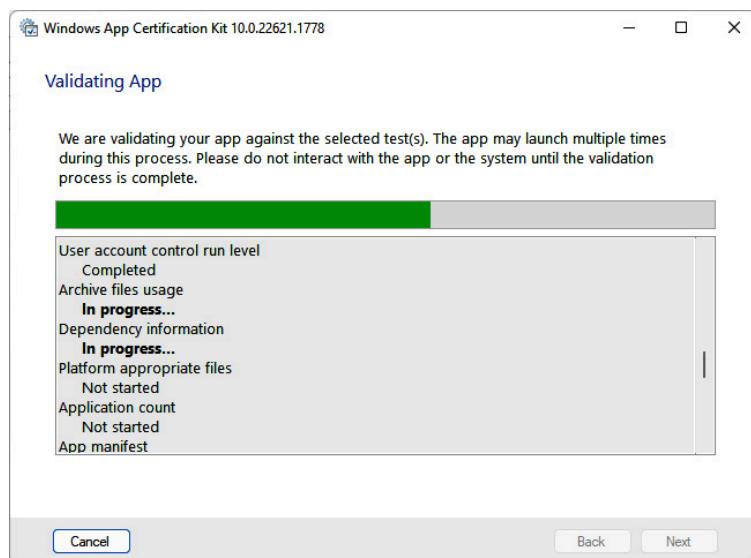


Figure 14.13 – The Microsoft Store application validation in process

9. The process should provide a **PASSED** result on the **View Final Report** page. Upon completion, you can click the **Click here to view the results** option to see which tests passed or failed. All failures should be addressed before continuing with the store submission. These tests will be run during the Microsoft Store approval process, and any failures here are likely to cause the submission to be rejected. For details on the tests and corrective actions that can be taken for failures, you can review this Microsoft Learn topic:

<https://learn.microsoft.com/windows/uwp/debug-test-perf/windows-app-certification-kit-tests>.

When you have a validated application ready to submit to the store, you can continue the process. Let's walk through how to submit a WinUI application through the Microsoft Partner Center website.

Uploading a package to the Store

In this section, we will walk through submitting a package created by Visual Studio to the Microsoft Store using the Partner Center dashboard. To do this, proceed as follows:

1. Start by logging in to the Partner Center with your Microsoft account at the following URL:
<https://partner.microsoft.com/dashboard/home>.
2. Click **Apps and Games**, and you will be taken to the **Overview** page to submit apps and games to Windows and Xbox.
3. You will see a list of reserved app names and submitted apps on your account here. If you do not yet have an app name reserved, you can click **New product | MSIX or PWA app**. I am going to select **My Media Collection** from my apps to submit an initial version of this application. The following screenshot illustrates how you would start to reserve a new name:

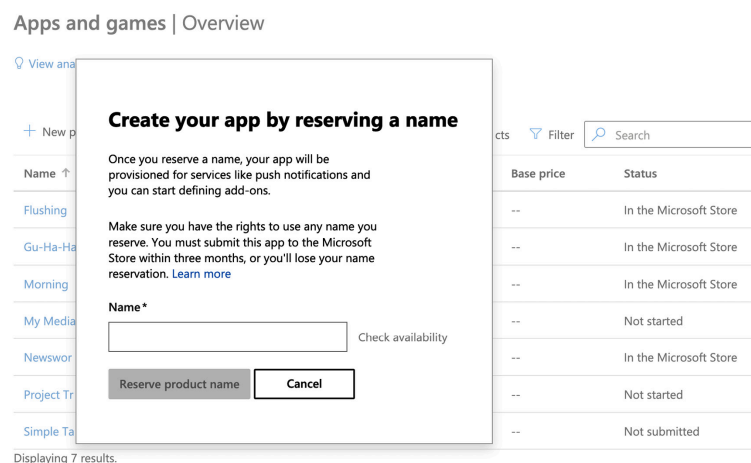


Figure 14.14 – Reserving a new name for your application

4. On the **Application overview** page for your selected app, click **Start your submission**, as illustrated in the following screenshot:

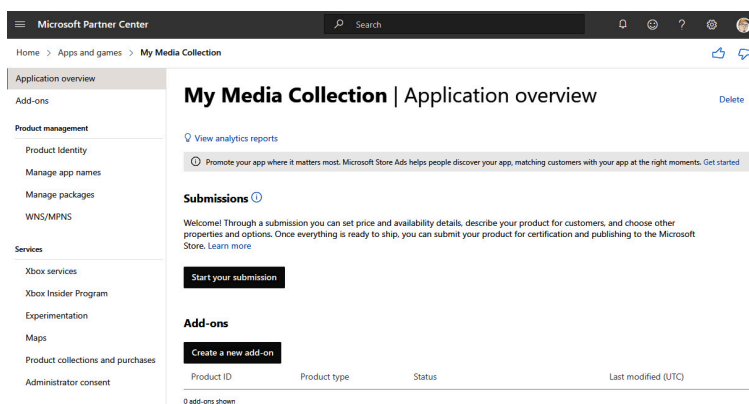


Figure 14.15 – Starting your new application submission

5. Begin the submission by selecting the **Pricing and availability** section.
6. We will keep all the default settings in this section, except for the **Base price** option. A selection must be made here. Choose the **Free** option or select a base price from the list. When you are done, select **Save draft**.
7. Then, select the **Properties** section. Select a category and sub-category, if necessary, and enter your **Support info** data. Enter any relevant data for your app in the **Display mode**, **Product declarations**, and **System requirements** sections, and click **Save**.

The **Properties** section can be seen in the following screenshot:

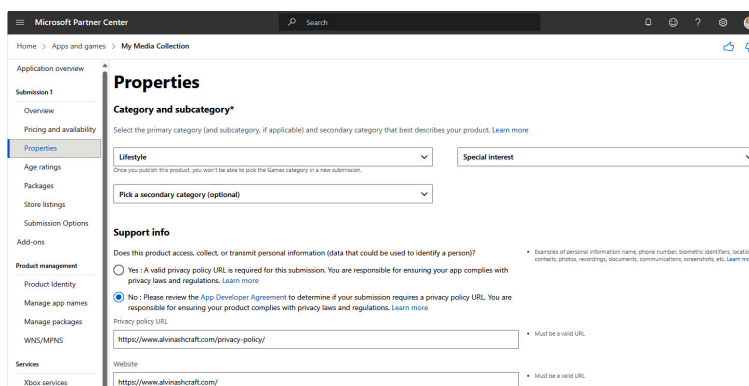


Figure 14.16 – The Properties section of the application submission

8. Complete the questionnaire on the **Age ratings** page and click **Pre-view ratings**. This page determines whether your app should be restricted to certain age groups, based on the data it collects or exchanges with other users. If everything looks good after generation, click **Save**, followed by **Continue**.
9. Select the **Packages** page. On this page, you can browse to your **.msixupload** or **.msix** file and upload it to the site for submission. Choose the compatible device families and click **Save**.
10. Then, select a language under the **Store listings** section. Supporting multiple languages and specifying them here makes it more likely that your application will be installed in different countries. Add an application description and at least one screenshot of your application. The remaining fields are optional, but the more you complete, the easier it will be for customers to find your app, and the more likely they are to try it. When everything is completed, click **Save**.

Some example language options are shown in the following screenshot:

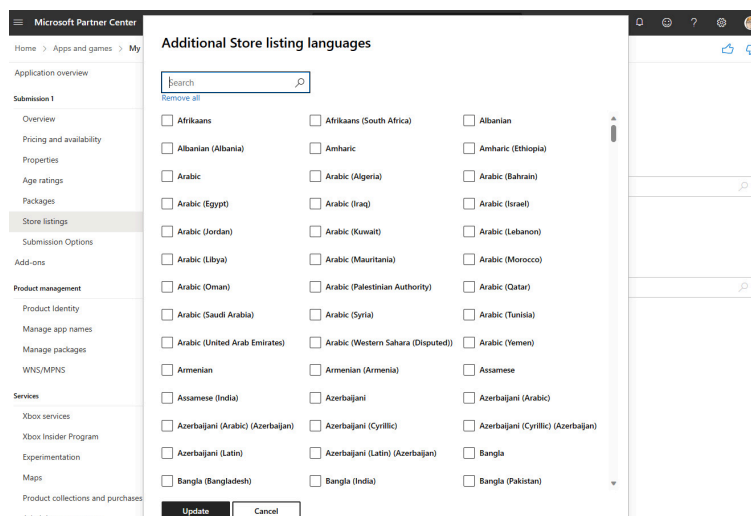


Figure 14.17 – Selecting languages for the application submission

11. Completing the **Submission options** page is optional. By default, your app will be published immediately after passing certification. I will select **Don't publish this submission until I select publish now** because I do not want this app to be available in the store until I make some additional changes.
12. Finally, click **Submit to the Store**. Your app will be submitted for certification. If it passes, it will be available in the Microsoft Store when you have indicated this on the **Submission options** page. If your app fails validation, you will receive a list of issues to address before attempting another submission.

Those are the basic steps to submit a new application to the Microsoft Store. For more detailed scenarios, and information about updates and add-ons, you can review the Microsoft Learn *app submission* documentation: <https://learn.microsoft.com/windows/apps/publish/publish-your-app/create-app-submission>.

Now, we will cover how to sideload applications in Windows with MSIX.

Sideloading WinUI applications with MSIX

In this section, we will create an MSIX package for a WinUI project and learn how to sideload it on Windows 10. When you sideload an application, you install it directly with the MSIX UI or PowerShell commands. This method of installation is important to understand, as it is frequently used by enterprises to distribute applications internally.

NOTE

You could also create a self-contained application package and use Xcopy deployment for distribution, but this method has some performance drawbacks, which we covered earlier in the chapter.

We are going to start by creating a package for sideloading.

Creating an MSIX package for sideloading

In this section, we will create a package for a WinUI project with Visual Studio. You can start by either opening an existing WinUI project or by creating a new, empty one. I have created a new project named **ProjectTracker**. Proceed as follows:

1. First, right-click the project in **Solution Explorer**, and select **Package and Publish | Create App Packages**.
2. On the **Select distribution method** screen of the **Create App Packages** window, leave the **Sideload** radio button and the **Enable automatic updates** checkbox selected. Click **Next**.
3. On the following page, select a signing method. Select **Yes, select a certificate**, and click the **Create** button. Here, you will create a self-signed certificate. By using a self-signed certificate, any users who install the app will need to trust your package and import the certificate from the MSIX package. We will explain this process in the following section, when we sideload the package. Enter a name and password for the certificate, as illustrated in the following screenshot:

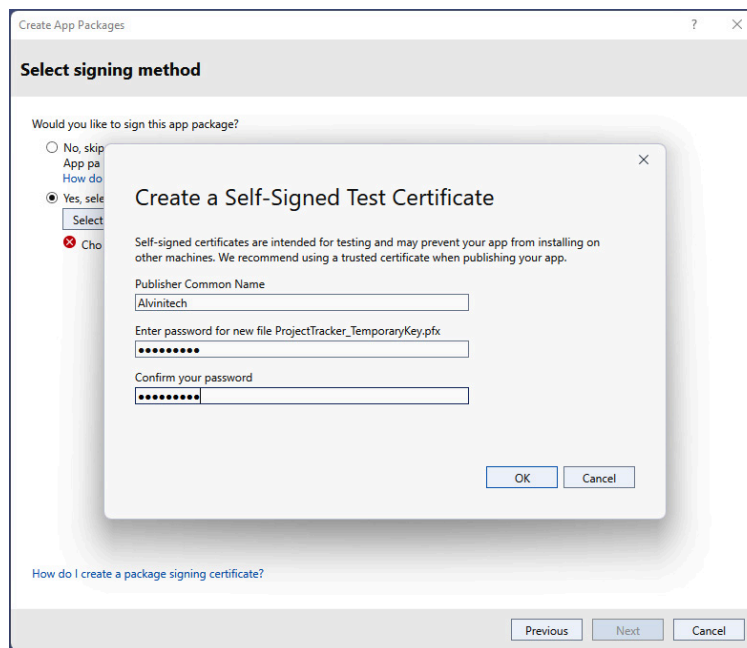


Figure 14.18 – Creating a self-signed certificate for the package

4. Select **Trust** to trust the certificate, and click **Next**.
5. Leave the default settings on the **Select and configure packages** page. Click **Next**.
6. Enter the installer path. This can be a local file path or a network location. Click **Create**. Your project will compile, and the package will be created in the specified location.

Now that we have an MSIX package for our project, we're ready to sideload it. Let's walk through this process.

Sideload an MSIX package

In this section, we will learn how to sideload a WinUI application with MSIX. We created a new MSIX package in the previous section. Navigate to the folder where the package was created inside your project's folder, and review the files in the package folder – named **ProjectTracker_1.0.0.0_x64_Test**, in my case. The following screenshot illustrates this:

Name	Date modified	Type	Size
Add-AppDevPackage.resources	9/23/2023 2:45 PM	File folder	
Dependencies	9/23/2023 2:45 PM	File folder	
Add-AppDevPackage	6/1/2023 6:50 PM	Windows PowerS...	37 KB
Install	6/1/2023 6:50 PM	Windows PowerS...	14 KB
ProjectTracker_1.0.0.0_x64	9/23/2023 2:45 PM	Security Certificate	1 KB
ProjectTracker_1.0.0.0_x64	9/23/2023 2:45 PM	MSIX File	48,648 KB
ProjectTracker_1.0.0.0_x64.msixsym	9/23/2023 2:45 PM	MSIXSYM File	10 KB

Figure 14.19 – Reviewing the package files

Now, let's look at the steps to sideload the application:

1. To install this package on another Windows device, start by copying the **Project Tracker_1.0.0.0_x64.msix** file to the machine. This file contains all of the necessary information and files to install the application.
2. Then, we need to install the self-signed certificate used to sign the package. You can install it by running the **Install.ps1 PowerShell** script, but you can also install it at the time it was created if you are installing it on the same machine. We will install it by right-clicking the MSIX file and selecting **Properties**.
3. Click the **Digital Signatures** tab, and select the certificate in the **Signature list** box, as illustrated in the following screenshot:

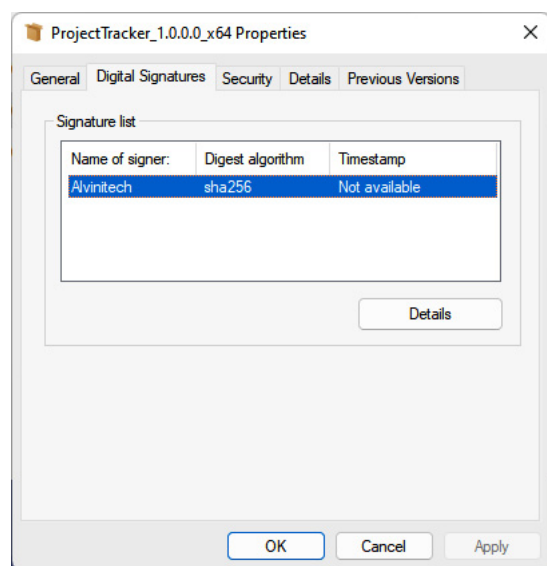


Figure 14.20 – Selecting the certificate in the MSIX package properties

4. Click **Details** to open the **Digital Signature Details** screen.
5. Click the **View Certificate** button. On the **Certificate** page that opens, click **Install Certificate**.

6. When completing **Certificate Import Wizard**, import the certificate to the local machine, and on the **Certificate Store** page, select **Place all certificates in the following store**, as illustrated in the following screenshot:

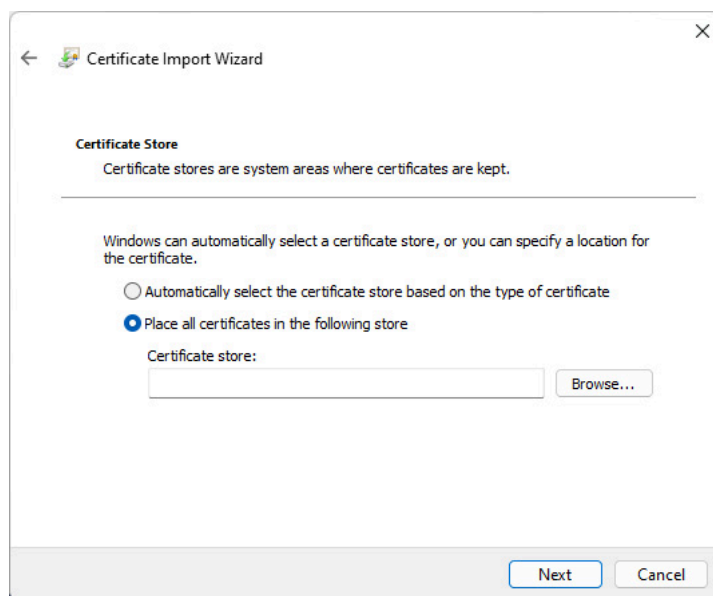


Figure 14.21 – Importing the package certificate

7. Click **Browse**, and select the **Trusted Root Certification Authorities** folder. Click **OK** on the dialog and click **Next** on the wizard.
8. After clicking **Finish**, the certificate will be imported. If all goes well, you will receive a message that the certificate was successfully imported. You can close the **Properties** page and continue with the package installation.
9. Now, double-click the **MSIX** package file. The installer will open a window with some of the application's manifest information. Click **Install**, and leave the **Launch when ready** checkbox selected, as illustrated in the following screenshot:

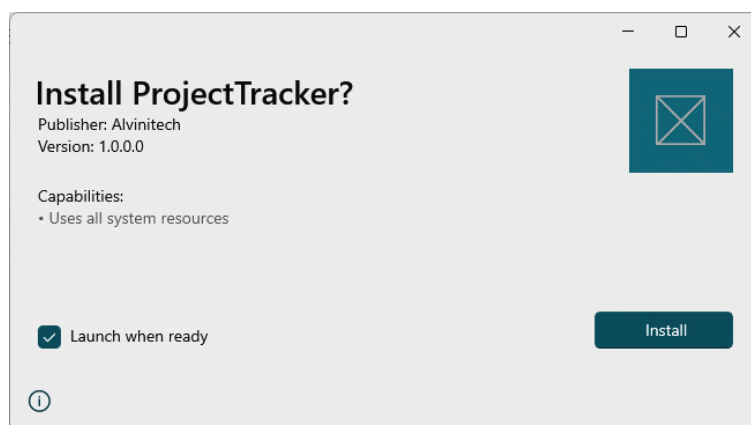


Figure 14.22 – Installing a trusted application from its MSIX package

The application will install and launch, and you're ready to go. Note that the package is trusted because we imported the certificate to the **Trusted Root Certification Authorities** folder.

This installation can be automated with PowerShell if the MSIX certificate is already trusted on the target machines. Use the **Add-AppPackage** command to install an MSIX package or MSIX bundle from a PowerShell prompt, as illustrated in the following command:

```
Add-AppPackage -path c:\Installers\ProjectTracker\ProjectTracker_1.0.0.0_x64.msix
```

If you have several packages to distribute, you can create a custom PowerShell script to iterate over all of the MSIX packages in a given folder. For more information about PowerShell scripting with MSIX, check out the following Microsoft Learn topic:

<https://learn.microsoft.com/windows/msix/desktop/powershell-msix-cmdlets>.

Let's wrap up and review what we have learned in this chapter.

Summary

In this chapter, we reviewed various methods of delivering WinUI applications to consumers. We learned the basics of MSIX packages and how to create packages to sideload our applications. We also covered the process of creating an account on Microsoft Partner Center to create application submissions on the Microsoft Store.

Then, we validated and submitted an MSIX application package to the Store. Finally, we learned how to manually sideload MSIX packages and how PowerShell can be leveraged to automate the sideloading process. These concepts will help you when you are ready to create your own WinUI applications for enterprise or consumer use.

This is the final chapter of our book. I hope that the concepts covered in each chapter will help you succeed in your quest to become a WinUI application developer.

Questions

1. What are some of the application installer formats that preceded MSIX?
2. Is MSIX only for UWP and WinUI apps?
3. In a WinUI project, which file contains the application manifest data?
4. Which command is used to install packages with Windows Package Manager?
5. How can you make your application available with WinGet?
6. What is the name of the online portal to submit applications to the Microsoft Store?
7. How many screenshots are required in a Microsoft Store listing?